Getting started with Nvidia Jetson Nano SD-CARD

In this guide, we will help you get started with balenaCloud by:

- Setting up your **Nvidia Jetson Nano SD-CARD** device and bringing it online on the balenaCloud dashboard.
- Deploying a *hello world* project on the device in your language of choice.
- Developing the sample project: making changes and testing them on the device in real-time.

Once you've completed this getting started guide to balena, you'll be equipped with the fundamentals needed to continue developing your application using balenaCloud and be on the path to deploying fleets of devices to production. If you are looking for definitions of certain terms, refer to the glossary.

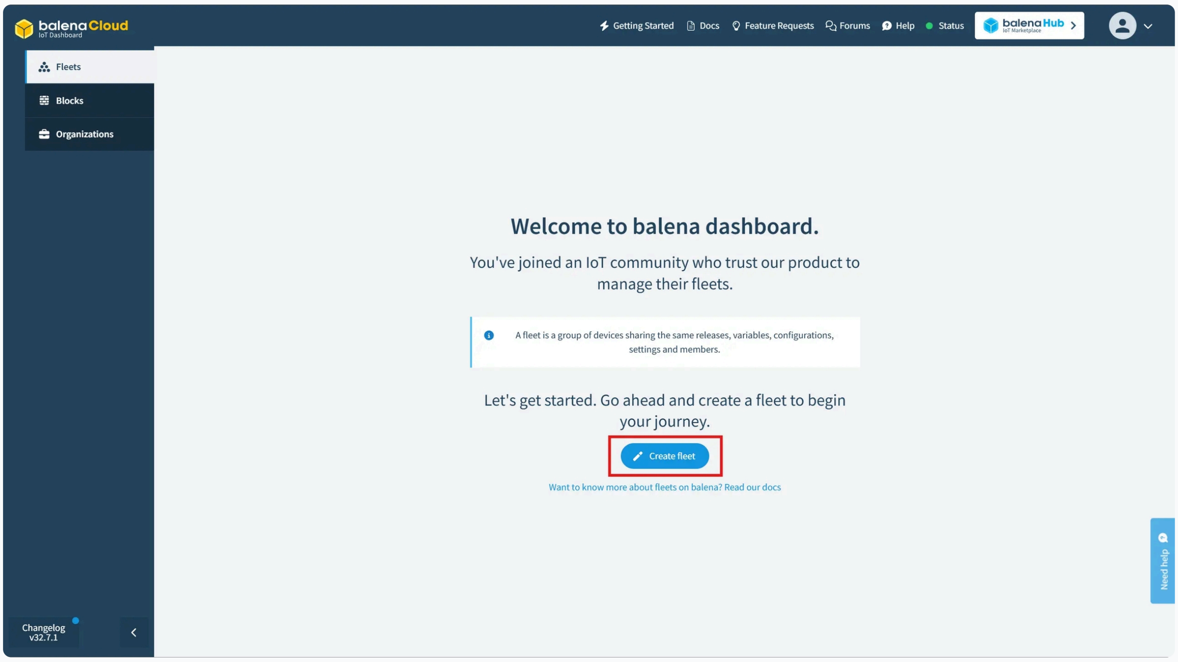
What you'll need

- A Nvidia Jetson Nano developer kit ↗
- A 16GB or larger microSD card. The speed class ↗ of the card also matters - UHS speed class I or above is recommended.
- An ethernet cable (the Jetson Nano developer kit does not include WiFi but it can be added via an optional mPCIe card or USB dongle).
- Compatible power supply - a 5V 4A barrel jack PSU is recommended, but you can also use a 5V 2.5A micro USB supply.
- A balenaCloud account ↗.

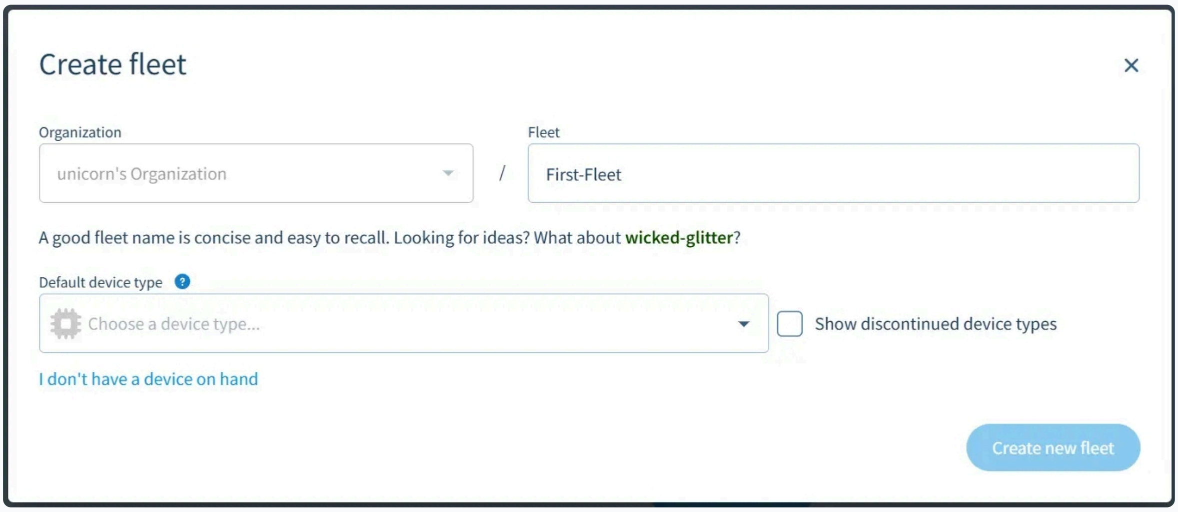
Create a fleet

A fleet is a group of devices that share the same architecture and run the same code. Devices are added to fleets and can be moved between fleets at any time.

To create your first fleet, log into your balenaCloud dashboard ↗ and click the **Create fleet** button.

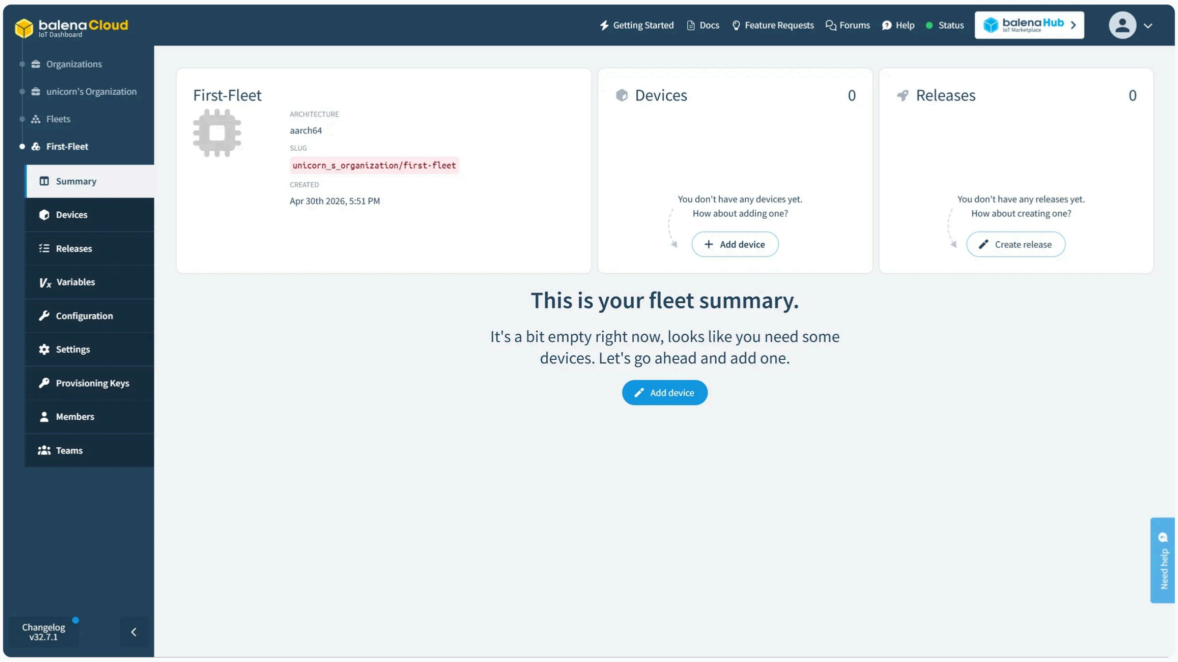


Enter a fleet name, select the **Nvidia Jetson Nano SD-CARD** device type, and click **Create new fleet**:

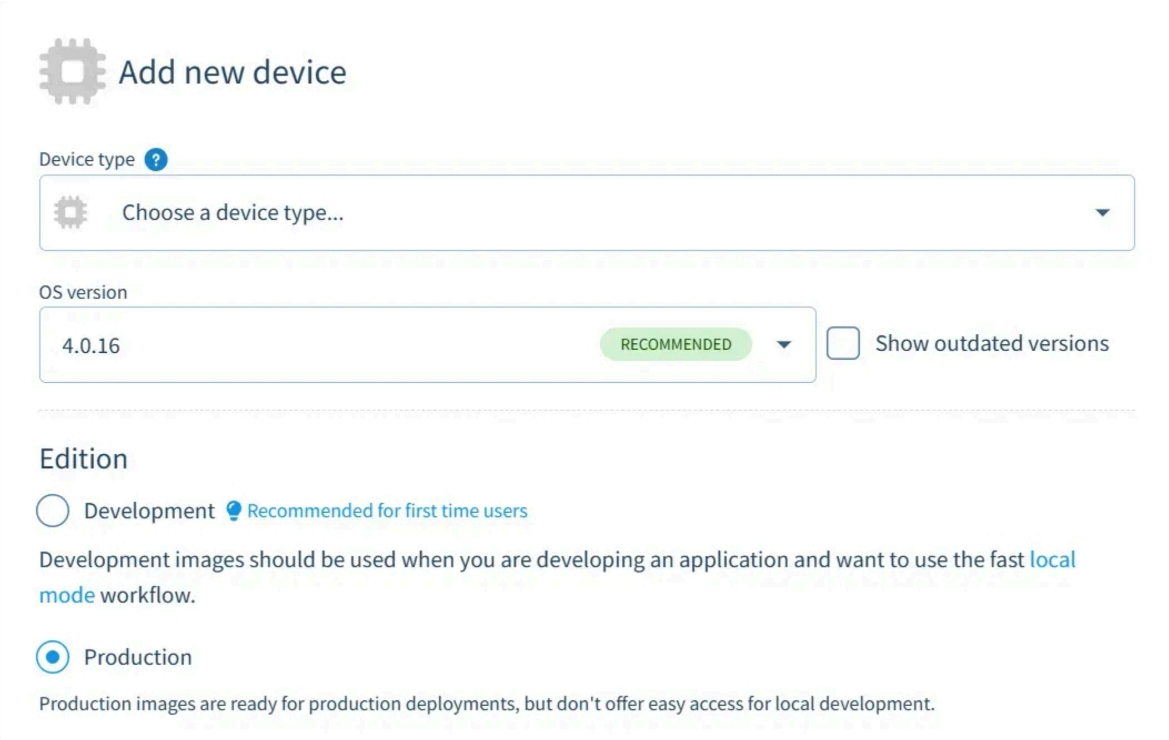


You'll then be redirected to the summary of the newly created fleet, where you can add your first Nvidia Jetson Nano SD-CARD.

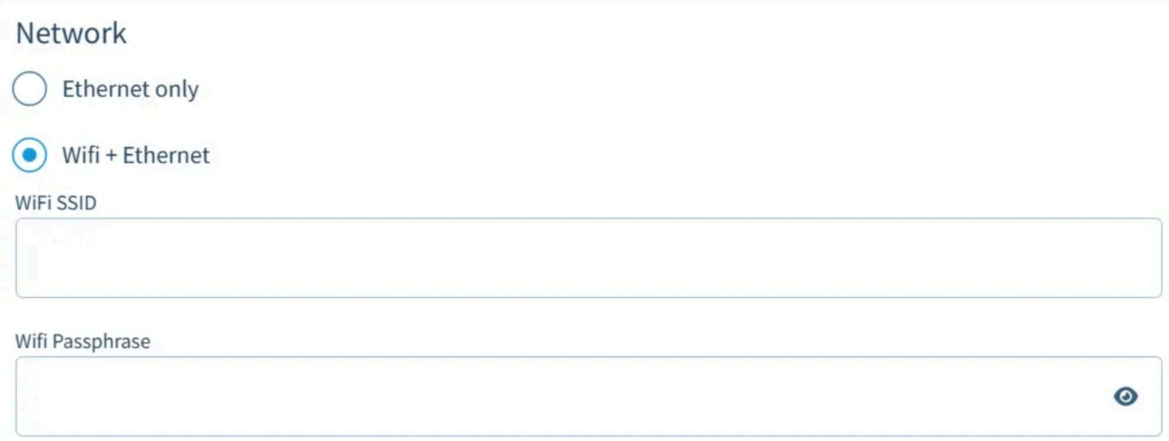
Add a device and download OS



balenaCloud builds a custom balenaOS image configured for Nvidia Jetson Nano SD-CARD which allows the device to provision and join the new fleet you created automatically. Start by clicking **Add device** on the fleet summary. Your device type will be preselected here since you already chose it when creating the fleet. Other device types of the same architecture can also be picked to join the fleet.

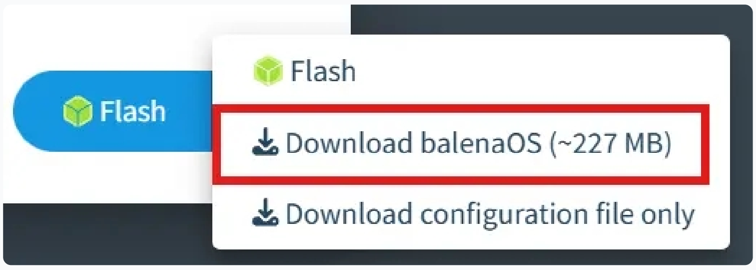


Select an OS type of *balenaOS*, and you will see a list of available balenaOS versions with the latest preselected. Choose a **Development** version of the OS. The production OS does not facilitate the development workflow we'll be using. Find out more about the differences between Development and Production images.



Select the type of network connection you'll be using: *Ethernet Only* or *Wifi + Ethernet*. A network connection is required to allow the device to connect to balenaCloud. Selecting *Wifi + Ethernet* allows you to enter a *Wifi SSID* and *Wifi Passphrase* which is then built into the image.

Click the arrow by the **Flash** button. You'll see several options.



Pick the option that works best for you:

Flash | Download balenaOS | Download configuration file only

If you have Etcher ↗ installed, you can just click the Flash button. This will automatically open Etcher with your image loaded. The contents of the image that gets loaded are the same as the contents of the image you would get from the **Download balenaOS** option. Once Etcher opens, proceed to Provision device.

This option is best for quickly flashing a device(s) in a single session. Once you close Etcher, the image will no longer be loaded in it.

Once you have chosen one of the options, the button should update to whichever option you have chosen. Click the button and proceed.

Provision device

Next, we will flash the downloaded image onto the device. To do so, follow the following steps:

- To provision Nvidia Jetson Nano SD-CARD, follow the instructions using our Jetson Flash tool ↗ to make the process more streamlined.

When complete, after a minute or two the device should appear on your balenaCloud dashboard ↗, and you should now be ready to deploy some code. If you are not able get the device to appear on the dashboard, then check out our troubleshooting guide for Nvidia Jetson Nano SD-CARD ↗ or try our support channels.

Install the balena CLI

Now that you have an `operational` device in your fleet, it's time to deploy some code. We will use the balena CLI for this. Follow the instructions below to install balenaCLI for the operating system available on your system. Skip the next part if you have balena CLI already installed.

OSX | Windows | Linux

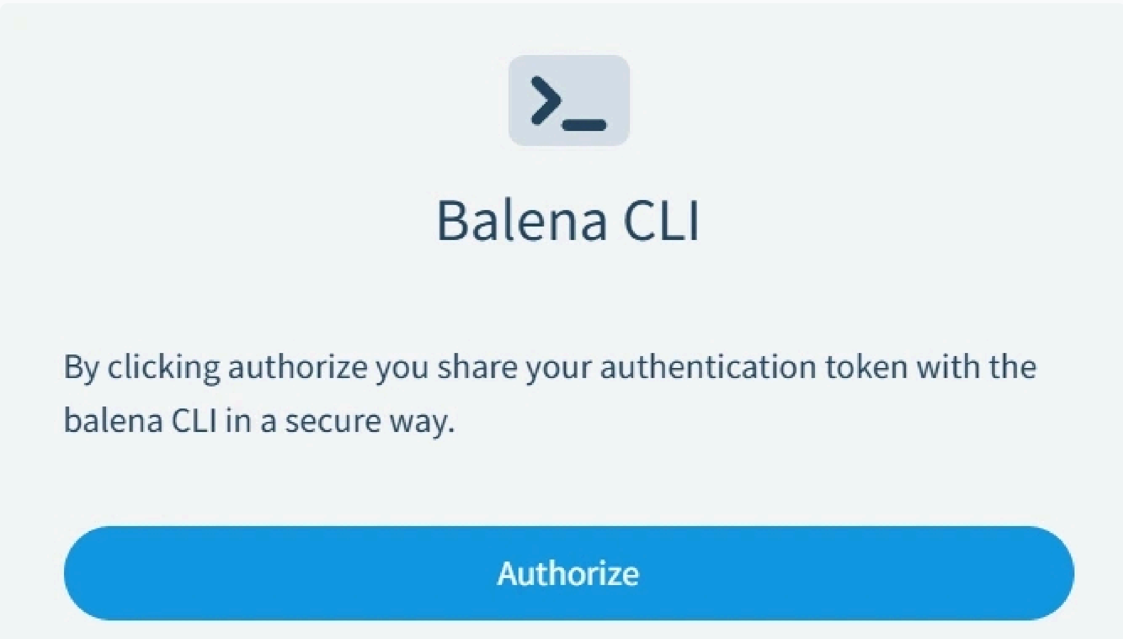
1. Download the CLI installer ↗.
2. Double click the downloaded file to run the installer and follow the installer's instructions.
3. To run balena CLI commands, open the Terminal app (Terminal User guide ↗).

After balena CLI is installed, login to your balena account using the `balena login` command on the terminal:

```
$ balena login
 _            _
| |__   __ _ | |  ____  _ __    __ _
| '_ \ / _` || | / __ \| '_ \  / _` |
| |_) | (_) || ||  ___/| | | || (_) |
|_.__/ \__,_||_| \____/|_| |_| \__,_|


Logging in to cloud.com
? How would you like to login? (Use arrow keys)
❯ Web authorization (recommended)
  Credentials
  Authentication token
  I don't have a balena account!
```

You will be asked to choose an authentication method, choose *Web authorization* which will bring up a web browser window that allows you to login to your balenaCloud account. Click the **Authorize** button, and head back to the terminal after the login successful message appears.



Create a release

After login, test the balena CLI by running the `balena fleet list` command, which should return information about the fleet you created in the previous step. Take a note of the fleet `NAME` as you'll need this in the next step to push the code to your device(s) in that fleet.

```
$ balena fleets
ID     NAME          SLUG                                   DEVICE
TYPE           DEVICE COUNT   ONLINE DEVICES
98264 First-Fleet   balena CLI/first-fleet    Nvidia Jetson Nano
SD-CARD    0              0
```

Node.js | Python | C++ | Rust | Go

A nice project to try is the [balena-nodejs-hello-world ↗]() project. It's a Node.js web server that serves a static page on port 80. To get started, [download the project ↗]() as a zipped file from GitHub, unzip it and open a terminal in the root of the extracted project directory.

To create a release, use the `balena push First-Fleet` command replacing `First-Fleet` with the name of your fleet. Ensure you are working from the root of the extracted project directory.

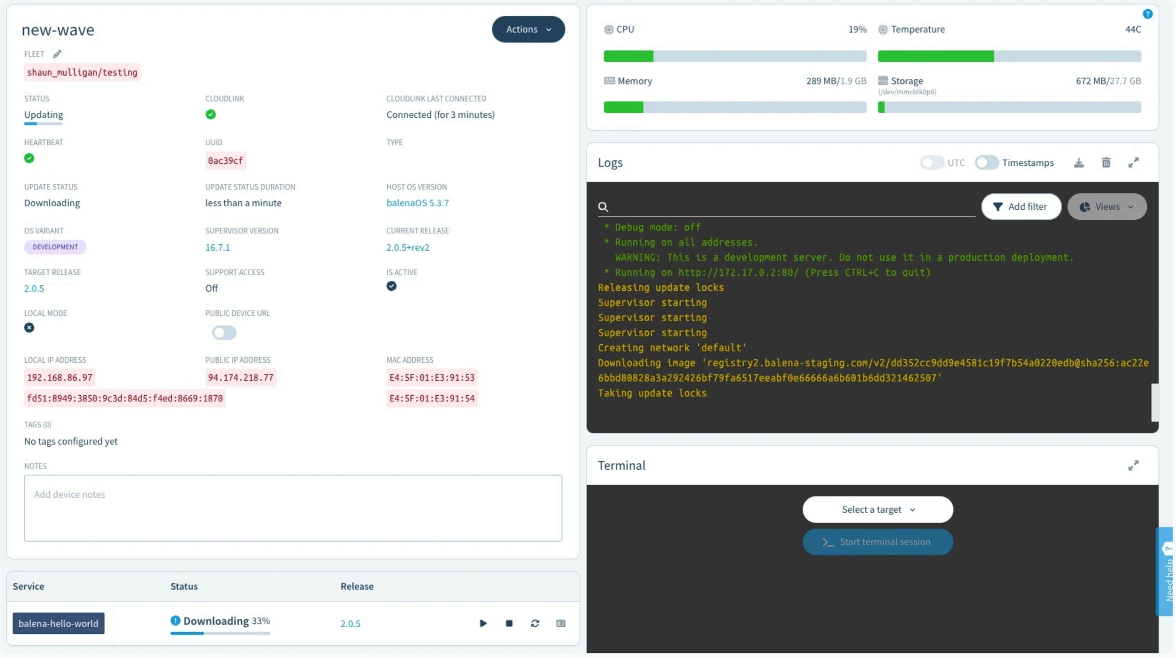
```
$ balena push First-Fleet
```

This command pushes the code to the balena builders, where it will be compiled, built, turned into a release, and applied to every device in the fleet.

You'll know your code has been successfully compiled and built when our friendly unicorn mascot appears in your terminal:

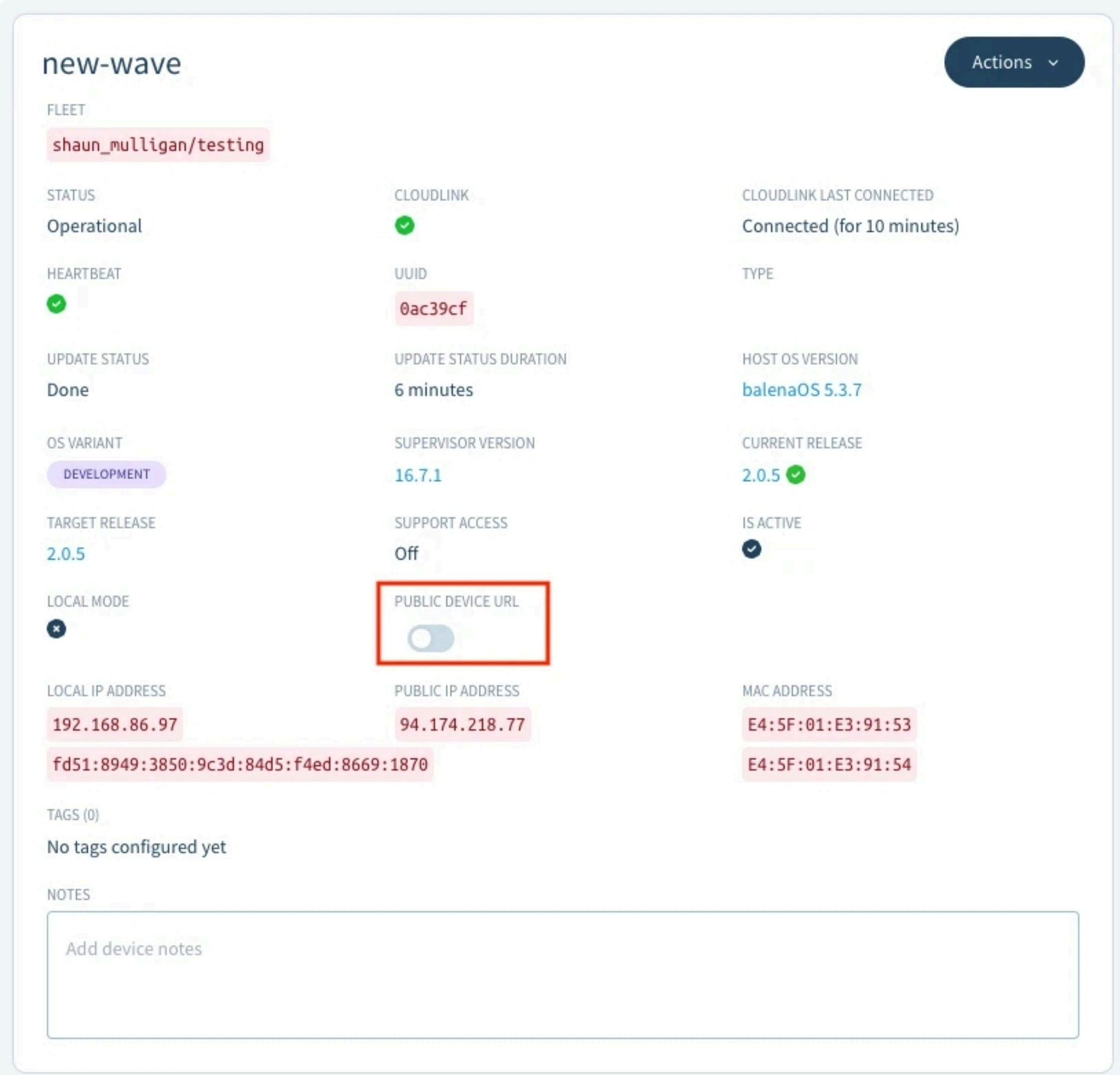
```
[hello-world]  Successfully built 51bd190f7530
[Info]       Generating image deltas from release 8acfdc579f7cb0fe424d1b800588b6f5 (id: 2186018)
[Success]    Successfully generated image deltas
[Info]       Uploading images
[Success]    Successfully uploaded images
[Info]       Built on builder05
[Success]    Release successfully created!
[Info]       Release: c0c593803588a304c173124827d96b99 (id: 2186339)
[Info]       ┌────────────────────┬────────────┬────────────┐
[Info]       │ Service            │ Image Size │ Build Time │
[Info]       ├────────────────────┼────────────┼────────────┤
[Info]       │ hello-world        │ 190.04 MB  │ 50 seconds │
[Info]       └────────────────────┴────────────┴────────────┘
[Info]       Build finished in 1 minutes, 4 seconds
                            \
                             \
                              \\
                               \\
                                >\/7
                            _.-(6'  \
                           (=___._/` \
                                )  \ |
                               /   / |
                              /    > /
                             j    < _\
                         _.-' :      ``.
                         \ r=._\        `.
                        <`\\_  \         .`-.
                         \ r-7  `-. ._  ' .  `\
                          \`,      `-.`7  7)   )
                           \/         \|  \'  / `-._
                                      ||    .'
                                       \\  (
                                        >\  >
                                    ,.-' >.'
                                   <.'_.''
                                     <'
```

The release will then be downloaded and started by all the devices in the fleet. You can see the progress of the device code updates on the device dashboard:

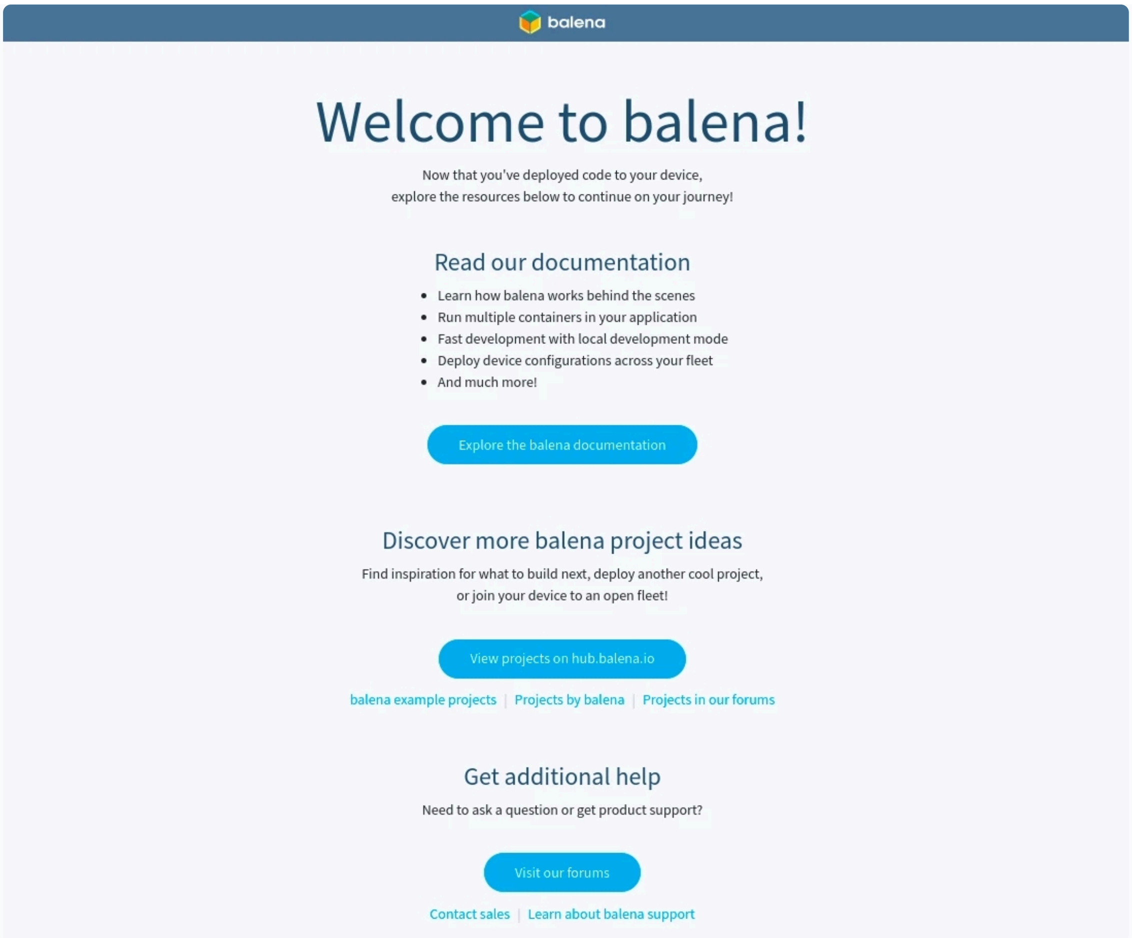


After the download, you should now have a web server running on your device and see some logs on your dashboard.

To give your device a public URL, click the *Public Device URL* toggle on the device dashboard. Public device URL allow you to serve content from the device to the world easily without configuration as long as the server is running on port 80.



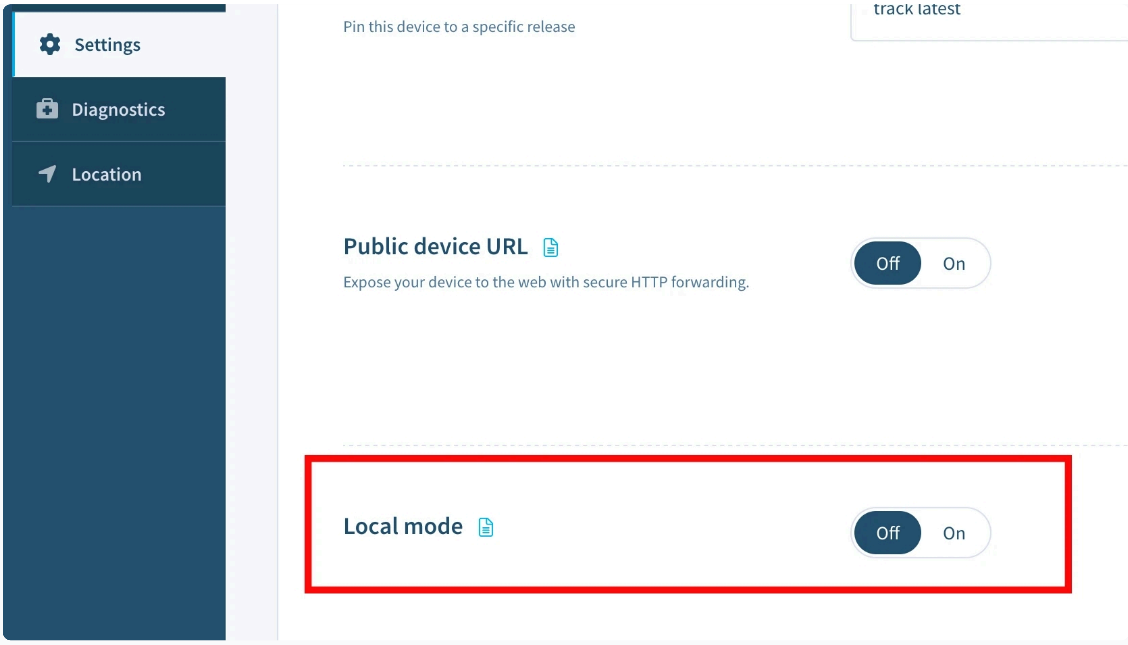
Follow the URL to view the welcome page with additional resources. Alternatively, you can point your browser to your device's local IP address to access the server running on your device. You can find the device's IP address on the device dashboard page. This is what you should be seeing.



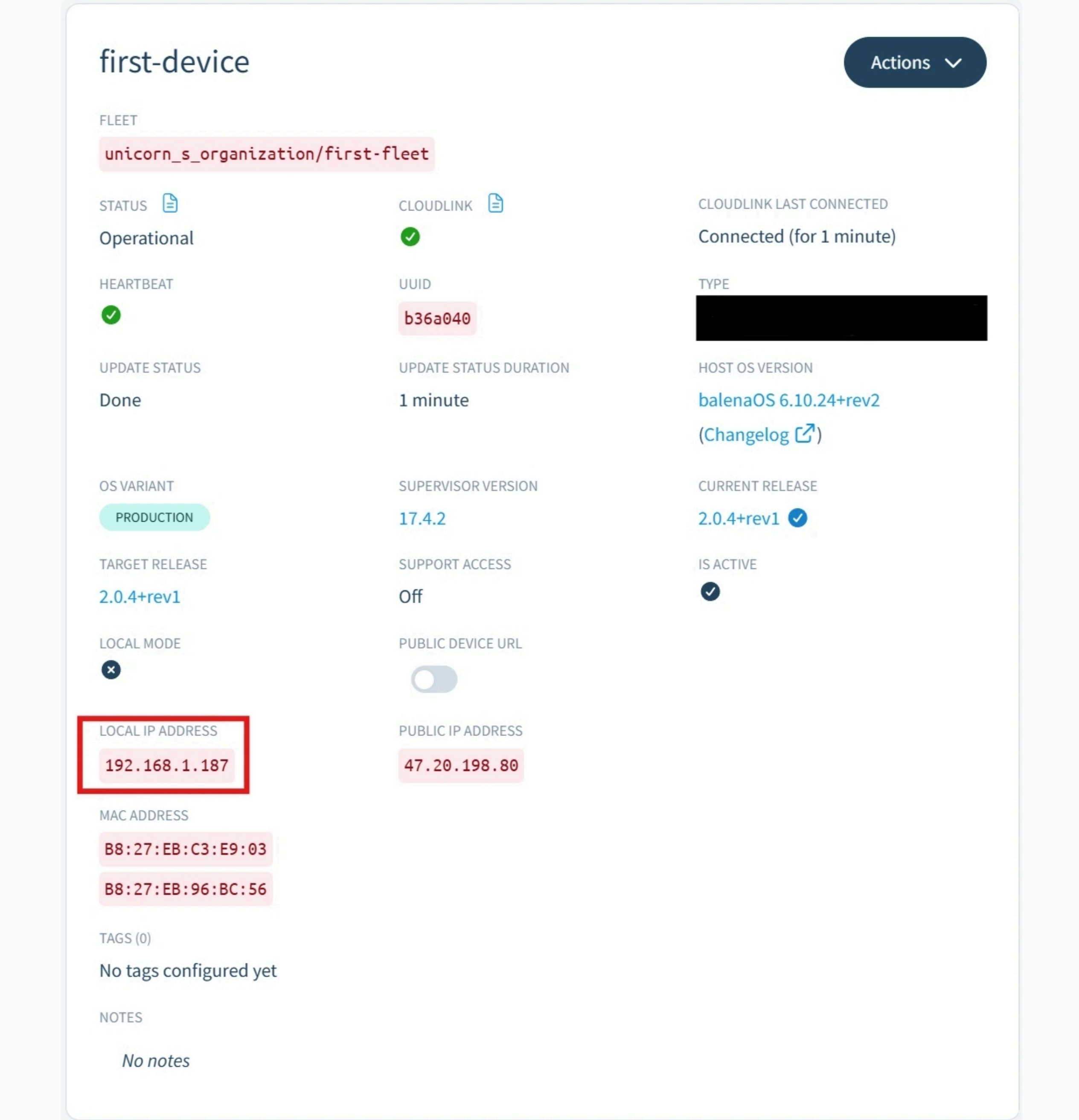
Developing your project

Now, let's try making some changes to this project and testing them right on the device. The project can be modified and pushed again using the same method as above, but since we are using a development version of the OS, we can enable *Local mode* and push directly to the device for a faster development cycle.

Activate local mode on the device via the dashboard.



Once enabled, you can now use `balena push` again, but this time we will push directly to the local IP address of the device obtained via the dashboard.




```
$ balena push 10.19.0.153
```

The same build process as before is carried out, but this time instead of using the balena builders, the build takes place locally on the device itself.

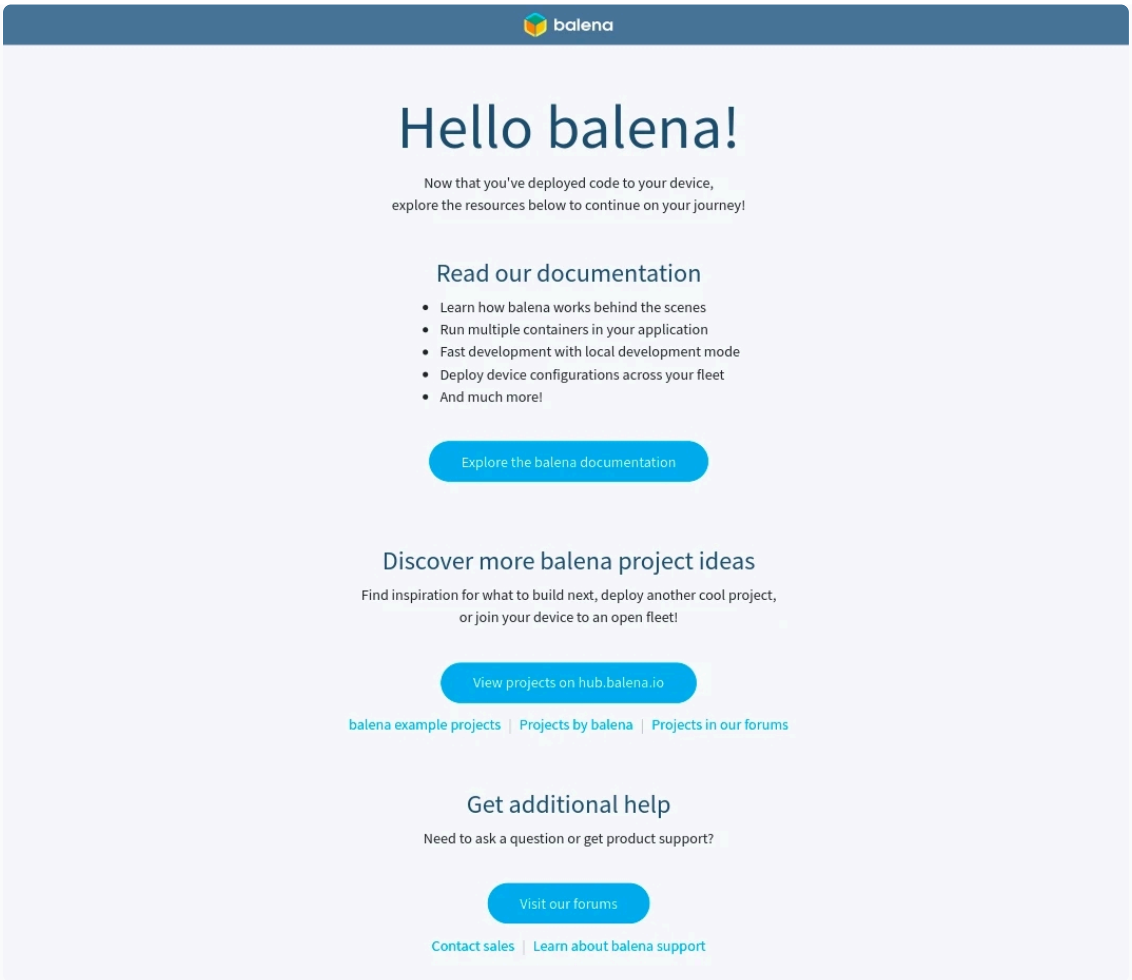
```
[Info]    Streaming device logs...
[Live]    Watching for file changes...
[Live]    Waiting for device state to settle...
[Logs]    [8/26/2021, 11:58:18 AM] Creating network 'default'
[Logs]    [8/26/2021, 11:58:19 AM] Installing service 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:58:20 AM] Installed service 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:58:20 AM] Starting service 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:58:23 AM] Started service 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:58:24 AM] [hello-world] Starting server on port 80
[Live]    Device state settled
```

The balena CLI will now watch for changes to all the files within the project, and automatically push changes to the device when detected. Let's try making a change to title of our balena welcome page. Navigate to the `index.html` file present in the `static` directory of the project. Open the file and change the title from `Welcome to balena!` to `Hello balena!` and save the file. After saving the changes, you can observe balena CLI automatically start rebuilding only the parts of the Dockerfile that has been changed.

```
[Live]    Detected changes for container hello-world, updating...

...
[Live]    [hello-world] Restarting service...
[Logs]    [8/26/2021, 11:59:01 AM, 2:42:32 AM] Service exited 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:59:04 AM, 2:42:35 AM] Restarting service 'hello-world sha256:...'
[Logs]    [8/26/2021, 11:59:05 AM, 2:42:36 AM] [hello-world] Starting server on port 80
```

When the rebuild is complete, take a look at the public device URL again to see your changes. The welcome page should have been updated with the new title.



Next steps

Once you've finished making your changes, disable local mode and the device will revert back to running the latest release that's on your fleet. To update your fleet with the latest changes you've just worked on, use `balena push <fleet name>` once more to create a new release with those changes.

When it's finished building the device(s) will update as before. Remember anything pushed to the fleet in this way can be applied to 10+ or 1000+ devices with no extra effort! To continue learning, explore parts of the guide in more detail:

- Learn more about local mode, which allows you to build and sync code to your device locally for rapid development.
- Develop an application with multiple containers to provide a more modular approach to fleet management.

- Manage your device fleet with the use of configuration, environment, and service variables.
- Find out more about the balena CLI ↗ and the functionality it offers.
- Visit our blog to find step-by-step tutorials for some classic balena projects ↗.
- To publish what you will build or have already built, head over to balenaHub ↗.
- If you find yourself stuck or confused, help is just a click away ↗.

Enjoy balenafying all the things!

Previous
Nvidia Jetson Nano eMMC

Next
Nvidia Jetson Orin Nano 8GB (SD) Devkit NVME

Last updated 2 days ago

Was this helpful?